

آشنایی با Git، GitHub و مفاهیم پایه وب

Git · GitHub · HTTP/HTTPS · URL · GET/POST · JSON/XML

۱. آشنایی با Git و GitHub

Git چیست و چه کاربردی دارد؟

Git یک سیستم کنترل نسخه توزیع شده (Distributed Version Control System) است که تاریخچه تغییرات فایل‌های یک پروژه را ذخیره می‌کند. با آن می‌توان به نسخه‌های قبلی برگشت، تغییرات را مقایسه کرد و چند نفر را همزمان روی یک پروژه کار داد. Git روی کامپیوتر خود شما اجرا می‌شود و برای کار کردن به اینترنت نیاز ندارد.

کاربردهای اصلی:

- ثبت تاریخچه تغییرات (commit) و امکان بازگشت به هر نسخه
- کار موازی روی قابلیت‌های جدید بدون خراب کردن کد اصلی (branch)
- ترکیب کار اعضای تیم (merge)

```
git init                # create a new repository
git add index.html       # stage a file
git commit -m "Add home page" # save a snapshot
git branch feature-login # create a new branch
git push origin main     # upload commits to remote
```

GitHub چیست و چه تفاوتی با Git دارد؟

GitHub یک سرویس میزبانی آنلاین برای مخزن‌های Git (Repository) است که امکاناتی مثل همکاری تیمی، Pull Request، Issues و GitHub Actions (CI/CD) را اضافه می‌کند.

ویژگی	Git	GitHub
ماهیت	ابزار (نرم‌افزار) خط فرمان	سرویس وب (شرکت مایکروسافت)
محل اجرا	روی کامپیوتر شما (محلی)	روی سرورهای ابری
نیاز به اینترنت	خیر	بله
هدف	مدیریت نسخه‌ها	اشتراک‌گذاری و همکاری
جایگزین‌ها	Mercurial, SVN	GitLab, Bitbucket

مثال: git مثل نرم‌افزار Word است و GitHub مثل Google Drive که فایل‌ها را آنلاین نگه می‌دارد و به اشتراک می‌گذارد.

۲. HTTP و HTTPS

HTTP چیست؟

HTTP (HyperText Transfer Protocol) **پروتکل ارتباطی** بین مرورگر (Client) و سرور (Server) برای انتقال صفحات وب و داده‌هاست. مرورگر یک Request می‌فرستد و سرور یک Response همراه با کد وضعیت (مثل 200 OK یا 404 Not Found) برمی‌گرداند. HTTP به صورت پیش فرض از پورت 80 استفاده می‌کند و داده‌ها را **رمزگذاری نشده (متن ساده)** منتقل می‌کند.

```
GET /index.html HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content-Type: text/html
```

HTTPS چیست؟

HTTPS (HTTP Secure) همان HTTP است که روی لایه رمزنگاری **TLS/SSL** اجرا می‌شود. داده‌ها رمزگذاری می‌شوند و هویت سرور با **گواهی دیجیتال (Certificate)** تأیید می‌شود. پورت پیش فرض آن 443 است.

تفاوت HTTP و HTTPS

مورد	HTTP	HTTPS
رمزگذاری	ندارد	دارد (TLS)
پورت	80	443
گواهی امنیتی	ندارد	نیاز دارد
امنیت	قابل شنود و دستکاری	امن در برابر شنود
نماد مرورگر	«Not secure»	قفل 

چرا امروزه بیشتر سایت‌ها از HTTPS استفاده می‌کنند؟

- **حفظ حریم خصوصی:** رمز عبور و اطلاعات کارت بانکی در مسیر قابل خواندن نیست (جلوگیری از حمله Man-in-the-Middle).
- **یکپارچگی داده:** محتوا در مسیر دستکاری یا تبلیغ تزریق نمی‌شود.
- **احراز هویت:** کاربر مطمئن می‌شود به سایت واقعی متصل است، نه سایت جعلی.
- **اعتماد و سئو:** مرورگرها سایت‌های HTTP را «نامن» نشان می‌دهند و گوگل HTTPS را در رتبه‌بندی ترجیح می‌دهد.
- **الزام فنی:** بسیاری از قابلیت‌ها مثل Geolocation، HTTP/2 و Service Worker فقط روی HTTPS کار می‌کنند.
- **رایگان شدن گواهی:** سرویس‌هایی مانند Let's Encrypt گواهی رایگان می‌دهند.

۳. URL

URL (Uniform Resource Locator) آدرس یکتای یک منبع روی اینترنت است. نمونه زیر را بررسی می‌کنیم:

https://www.example.com:8080/products/shoes?color=red&size=42#reviews

بخش	مقدار در مثال	توضیح
Scheme / Protocol	https	پروتکل ارتباط (http, https, ftp و ...)
Subdomain	www	زیردامنه
Domain Name	example.com	نام دامنه (example نام سایت و .com پسوند TLD)
Port	8080	شماره پورت سرور (اختیاری؛ پیش‌فرض 443 = HTTPS)
Path	/products/shoes	مسیر منبع روی سرور
Query String	?color=red&size=42	پارامترهای key=value که با & جدا می‌شوند
Fragment	#reviews	پرش به بخشی از صفحه (در مرورگر پردازش می‌شود و به سرور نمی‌رود)

۴. متدهای GET و POST

GET چیست؟

متدی برای دریافت (خواندن) اطلاعات از سرور. پارامترها در URL قرار می‌گیرند، درخواست تغییری در داده‌های سرور ایجاد نمی‌کند (Safe و Idempotent) و می‌تواند Cache و Bookmark شود.

POST چیست؟

متدی برای ارسال داده به سرور جهت ایجاد یا پردازش یک منبع. داده‌ها در Body درخواست می‌روند و در URL دیده نمی‌شوند. معمولاً روی سرور تغییر ایجاد می‌کند و Cache نمی‌شود.

تفاوت GET و POST

مورد	GET	POST
هدف	دریافت داده	ارسال/ایجاد داده
محل داده	در URL (Query String)	در Body درخواست
امنیت	کمتر (داده در URL و تاریخچه مرورگر می ماند)	بیشتر (داده در URL نیست؛ ولی بدون HTTPS رمز نمی شود)
محدودیت حجم	محدود به طول URL	تقریباً نامحدود
Cache / Bookmark	بله	خیر
تکرار درخواست	بی خطر (Idempotent)	ممکن است چند بار ثبت شود

هر کدام برای چه کارهایی استفاده می شوند؟

- **GET:** باز کردن صفحه، جستجو، نمایش لیست محصولات، دریافت اطلاعات از API
- **POST:** ثبت نام و ورود (Login)، ارسال فرم، آپلود فایل، ثبت سفارش، ایجاد پست جدید

مثال واقعی

مثال GET (جستجو در Google): مرورگر با این آدرس، نتایج جستجو را می گیرد:

```
GET https://www.google.com/search?q=git+tutorial
```

مثال GET در API (GitHub API): دریافت اطلاعات یک کاربر:

```
GET https://api.github.com/users/octocat
```

مثال POST (ایجاد Issue در GitHub API):

```
POST https://api.github.com/repos/octocat/hello/issues
```

```
Content-Type: application/json
```

```
{ "title": "Bug in login page", "body": "Button does not work" }
```

مثال POST در وبسایت: هنگام فشردن دکمه Login در یک سایت، نام کاربری و رمز در Body به `/login` فرستاده می شود:

```
POST /login
```

```
Content-Type: application/x-www-form-urlencoded
```

```
username=ali&password=*****
```

JSON چیست؟

JSON (JavaScript Object Notation) یک **قالب سبک متنی** برای ذخیره و تبادل داده است که به صورت جفت‌های `key: value` نوشته می‌شود. انواع داده: رشته، عدد، `null`، `true/false`، آرایه `[ ]` و شیء `{ }`.

XML چیست؟

XML (eXtensible Markup Language) یک **زبان نشانه‌گذاری** برای توصیف داده‌هاست که با **تگ‌های تودرتو** ساخته می‌شود و می‌توان تگ‌ها را آزادانه تعریف کرد. از `Attribute`، `Namespace` و `Schema` (پشتیبانی می‌کند).

کاربردها

- **JSON**: پاسخ‌های REST API، ارتباط فرانت‌اند و بک‌اند، فایل‌های تنظیمات (مانند `package.json`), دیتابیس‌های NoSQL (مثل MongoDB)
- **XML**: سرویس‌های SOAP، فایل‌های تنظیمات (مثل `AndroidManifest.xml`، `pom.xml`), فید RSS، اسناد Office (`.docx`), تصاویر SVG

تفاوت JSON و XML

مورد	JSON	XML
ساختار	جفت <code>key/value</code> ، آبجکت و آرایه	تگ‌های باز و بسته تودرتو
حجم و سرعت	کم‌حجم‌تر و سریع‌تر	پرحجم‌تر (به‌خاطر تگ‌های تکراری)
خوانایی	ساده و مختصر	طولانی‌تر
آرایه	پشتیبانی بومی	با تکرار تگ
کامنت	ندارد	دارد (<code><!-- --></code>)
Attribute / Namespace / Schema	ندارد	دارد (XSD)
تجزیه (Parse) در JavaScript	بسیار ساده (<code>JSON.parse</code>)	نیاز به پارسر (DOM)
کاربرد رایج	API‌های مدرن	سیستم‌های سازمانی و قدیمی

اطلاعات یک Student با JSON

```
{
  "student": {
    "id": 1001,
    "firstName": "Ali",
    "lastName": "Ahmadi",
    "age": 21,
    "major": "Computer Engineering",
    "gpa": 17.45,
    "isActive": true,
    "courses": ["Web Programming", "Database", "Algorithms"],
    "address": {
      "city": "Tehran",
      "country": "Iran"
    }
  }
}
```

اطلاعات همان Student با XML

```
<?xml version="1.0" encoding="UTF-8"?>
<student id="1001">
  <firstName>Ali</firstName>
  <lastName>Ahmadi</lastName>
  <age>21</age>
  <major>Computer Engineering</major>
  <gpa>17.45</gpa>
  <isActive>true</isActive>
  <courses>
    <course>Web Programming</course>
    <course>Database</course>
    <course>Algorithms</course>
  </courses>
  <address>
    <city>Tehran</city>
    <country>Iran</country>
  </address>
</student>
```

همان‌طور که دیده می‌شود، نسخه JSON کوتاه‌تر است، ولی XML برای هر مقدار تگ باز و بسته دارد.